

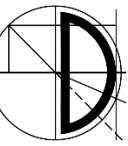


Grundlagen der Programmierung

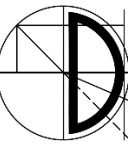
Kapitel 4: Anweisungen und Ablaufsteuerung (Teil 1)

Prof. Dr. Samuel Kounev, Daniel Grillmeyer M.Sc.

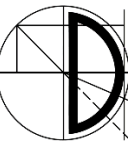
- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich



- Warum benötigen wir eine Ablaufsteuerung und was versteht man darunter?
- Anweisungen
- Blöcke und ihre Struktur
- Entscheidungsanweisungen
 - Die `if`-Anweisung
 - Die `switch`-Anweisung



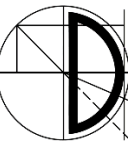
- Bisher bekannte Arten von Anweisungen
 - Deklaration z.B. `int a;`
 - Wertzuweisung z.B. `a = 5;`
 - Methodenaufruf z.B. `IO.println(a);`
 - Leere Anweisung z.B. `;`
- Hierdurch können wir nur rein lineare Programmstrukturen erzeugen
- Dieses Vorlesungskapitel beschäftigt sich mit weiteren Strukturierungsmöglichkeiten



- Ein **Block** ist eine zusammengefasste Folge von Anweisungen
- Sie dienen der Strukturierung von Programmen
- In Java und C/C++ werden Blöcke durch geschweifte Klammern begrenzt:

```
{  
    <Anweisung>  
    ...  
    <Anweisung>  
}
```

- Blöcke können geschachtelt sein



- Variablen sind nur bis zum Ende des Blocks gültig, in dem sie definiert wurden

j ist
unbekannt

j ist
bekannt

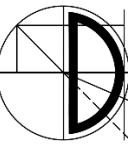
j ist
unbekannt



```
{  
    int i = 2;  
    {  
        int j = 3;  
        ...  
    }  
    ...  
    ...  
}
```



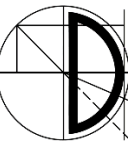
i ist
bekannt



- Die bisher (z.B. ohne Verzweigungen und Schleifen) erzeugbaren Programme haben eine sehr einfache Gestalt:

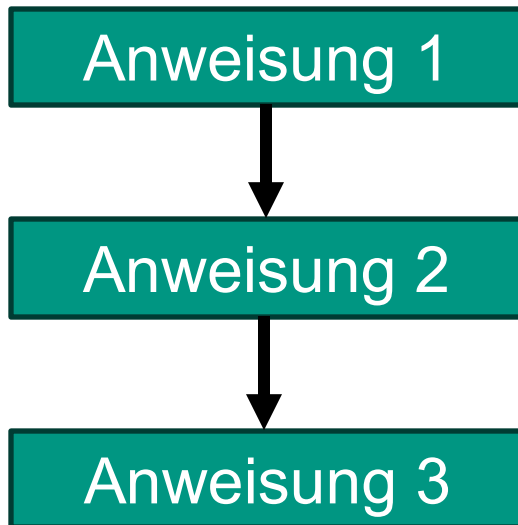
```
public class Bezeichner {  
    public static void main (String [] args) {  
        <Anweisung>  
        <Anweisung>  
        ...  
        <Anweisung>  
        <Anweisung>  
    }  
}
```

- Anweisungen können hier auch durch Blöcke ersetzt werden

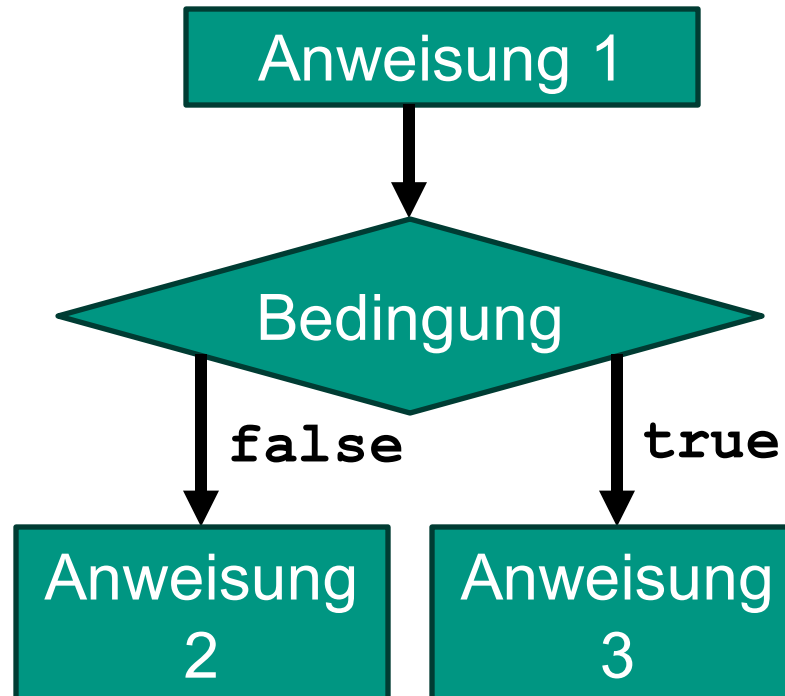


- Verschiedene Abläufe visualisiert als Programmablaufpläne (PAP)

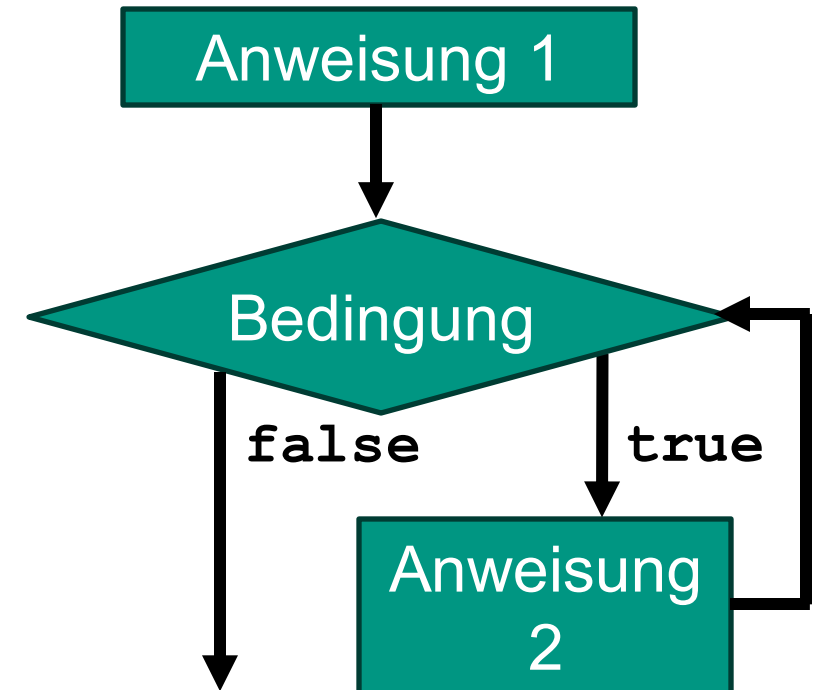
Linearer Ablauf

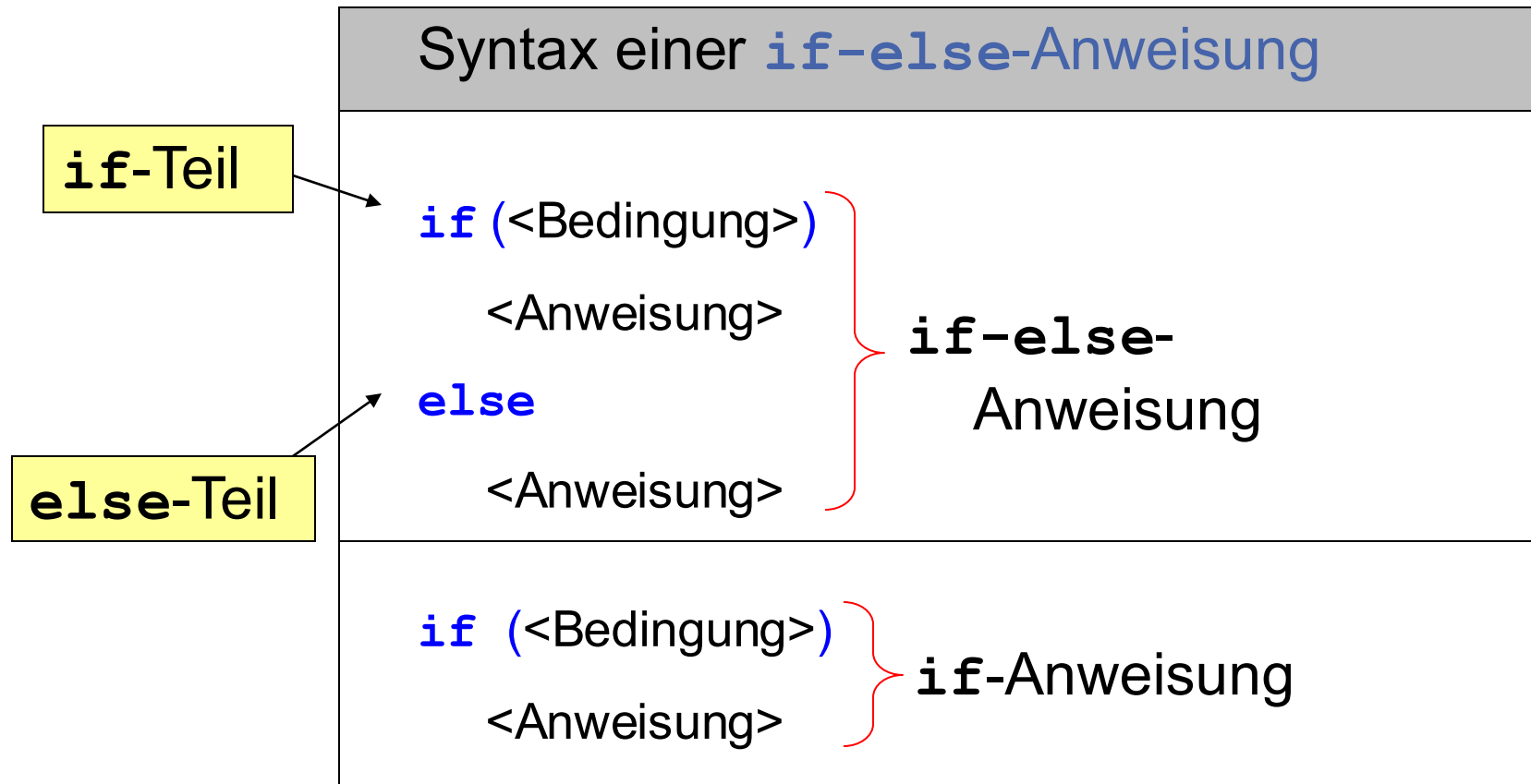
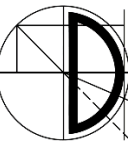


Verzweigung

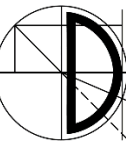


Schleife



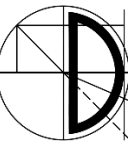


<Bedingung> = Boolescher Ausdruck

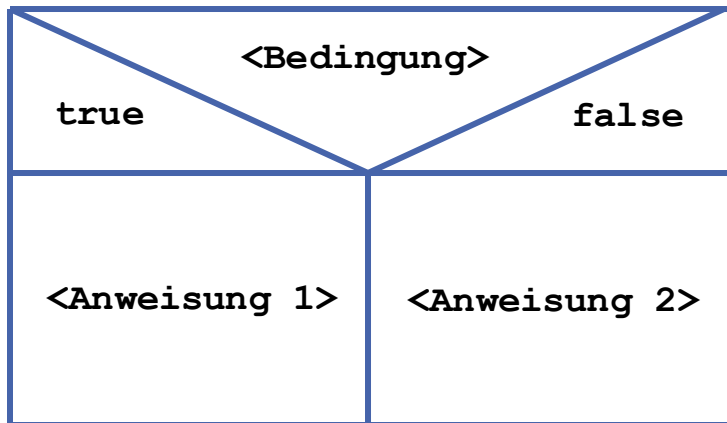


Die Anweisungen können durch Blöcke ersetzt werden

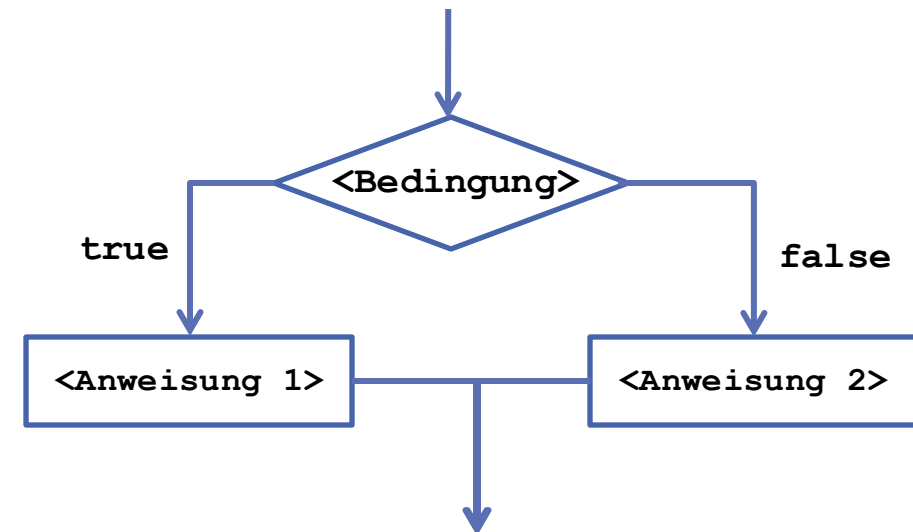
```
if (<Bedingung>) {  
    <Anweisung>  
    ...  
    <Anweisung>  
}  
else {  
    <Anweisung>  
    ...  
    <Anweisung>  
}
```



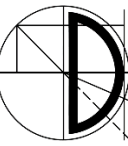
- Zunächst wird die Bedingung ausgewertet
- Ist sie **wahr (true)**, so führt man den `if`-Teil aus und überspringt den `else`-Teil
- Ist sie **falsch (false)**, so überspringt man den `if`-Teil und führt nur den `else`-Teil aus



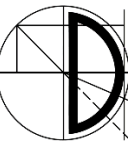
Struktogramm



Programmablaufplan (PAP)



```
int i = 6;  
if (i % 2 == 0)  
    IO.println("i ist eine gerade Zahl");  
else  
    IO.println("i ist eine ungerade Zahl");
```



Übung: Zu welchem `if` gehört das **else**?

```
int a;  
int b;  
...           // Anweisungen
```

```
if (a > 0)
```

```
    if (b > a)
```

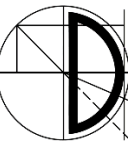
```
        IO.println("b > 1");
```

```
else
```

```
    IO.println("Fehler: a ist <= Null");
```

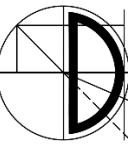
zum zweiten **if**

Klammern können Semantik
präzisieren



Best practice: Blöcke verwenden!

```
int a;  
int b;  
...           // Anweisungen  
if (a > 0) {  
    IO.println("a ist > Null");  
} else {  
    IO.println("a ist <= Null");  
}
```

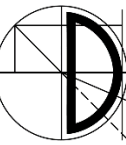


Beispiel: Wo endet diese if-else-Anweisung?
Welches Problem taucht hier auf?

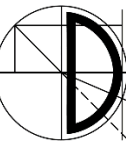
```
int x = 0;  
if (x == 0) {  
    IO.println("x ist gleich 0");  
}  
else  
    IO.println("x ist ungleich 0");  
    IO.println("1/x liefert " + 1/x);  
    IO.println("Division durchgefuehrt");
```

Hier

Problem: Division durch 0. Es fehlen Klammern!



```
Scanner input = new Scanner(System.in) ;
IO.println("Punktzahl: ") ;
int punkte = input.nextInt() ;
String note;
if (punkte > 50) {
    note = "sehr gut";
} else if (punkte > 25) {
    note = "ausreichend";
} else {
    note = "nicht bestanden";
}
IO.println("Note: " + note) ;
```

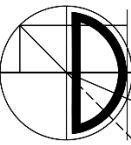
```
// (e) Nicht compilierbar!  
int u = 5, v = 2, w = 4;  
int z;  
if (u = v + w)  
    z = 27;  
else  
    z = z + 27;
```

Zuweisung anstelle von Vergleich!
Dadurch `int`-Wert in der Klammer.

Die Variable `z` ist nicht initialisiert!

```
// (f) Compilierbar, aber logisch problematisch!  
int z = 100;  
boolean bu = true, bv = false, bw = false;  
if (bu = bv && bw)  
    z = 42;  
else  
    z = z + 42;  
System.out.println("z = " + z);  
System.out.println("bu = " + bu);
```

Zuweisung anstelle von Vergleich!
Dadurch ist der Wert in der Klammer
durch den neuen Wert von `bu` gegeben.



Im Gegensatz zur `if`-Anweisung erlaubt die `switch`-Anweisung eine Auswahl unter mehreren Alternativen

Syntax `switch`-Anweisung

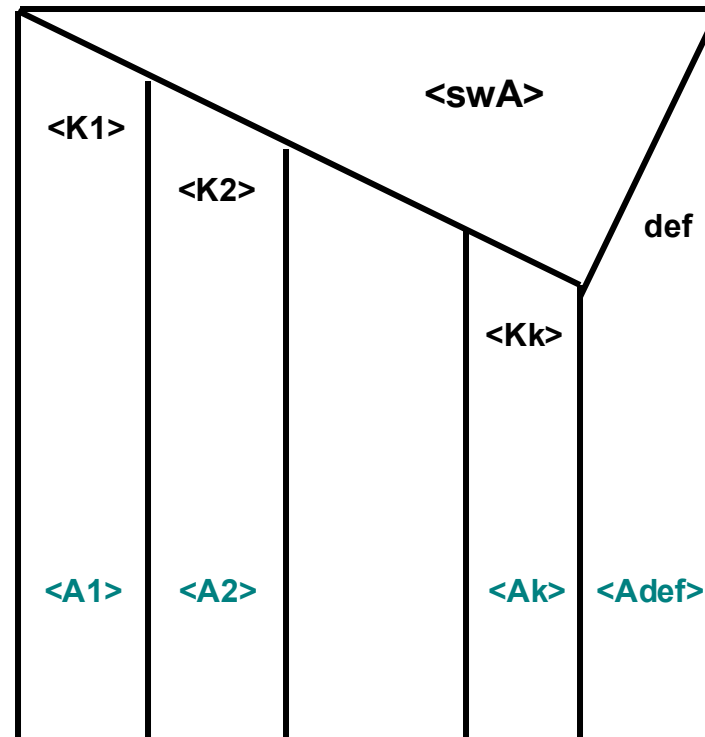
```
switch (<switch-Ausdruck>) {  
    case <Konstante> : <Anweisungsfolge>; break;  
    case <Konstante> : <Anweisungsfolge>; break;  
    ...  
    case <Konstante> : <Anweisungsfolge>; break;  
    default : <Anweisungsfolge>  
}
```

Semantik: Berechne Wert des `switch`-Ausdrucks und prüfe der Reihe nach die Konstanten bis eine Übereinstimmung gefunden wird. Stimmt der Wert erstmals mit der *i*-ten Konstante überein, so gelangt *i*-te Anweisungsfolge zur Ausführung.

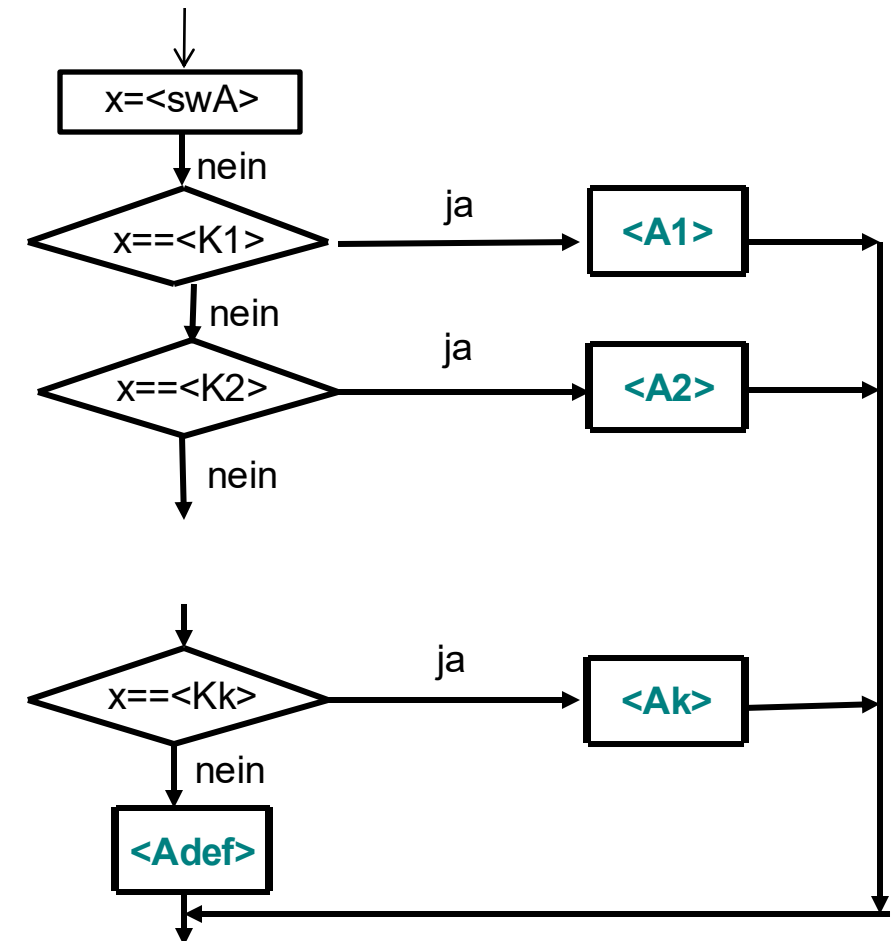
Der `default`-Zweig ist optional. Es darf höchstens einen `default`-Zweig geben.
`break` ist optional.

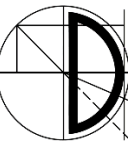
```
switch (<swA>) {  
    case <K1>: <A1> break;  
    case <K2> : <A2> break;  
    ...  
    case <Kk> : <Ak> break;  
    default : <Adef>  
}
```

Struktogramm



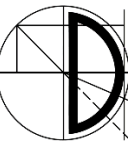
Programmablaufplan (PAP)



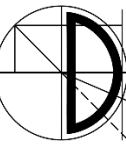


- Befindet sich ein **break** hinter der betreffenden Anweisung, so wird die **switch**-Anweisung beendet, *andernfalls* werden auch die weiteren Anweisungsfolgen ausgeführt (*bis zum nächsten break*).
- Stimmt der Wert des **switch**-Ausdrucks mit keinem der Werte hinter dem Schlüsselwort **case** überein, so wird die Anweisung hinter dem **default** ausgeführt.
- Danach wird die **switch**-Anweisung beendet.

Beachte: Vergessene **break**-Anweisungen führen oft zu schwer auffindbaren Fehlern!



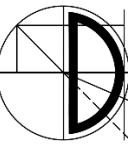
- Der **switch**-Ausdruck muss einen der folgenden Datentypen zurückliefern:
 - **byte**
 - **short**
 - **int**
 - **char**
 - **String**
 - **Character, Byte, Short, Integer** (kommt in Kapitel 15)
 - **enum** (kommt in Kapitel 17)
- Kein Wert der Konstanten der **case**-Zweige darf doppelt auftreten. Ihre Ordnung ist jedoch egal.



```
Scanner input = new Scanner(System.in);  
char t = input.nextChar();  
int s = 7;  
switch (t) {  
    case 'X':  
        s = 25;  
        break;  
    case 'Y':  
        s = 40;  
        break;  
}  
IO.println("s = " + s);
```

Eingabe:	Ausgabe:
X	s = 25
Y	s = 40
x	s = 7
r	s = 7

Beispiel: switch-Anweisung



```
Scanner input = new Scanner(System.in);
System.out.print("a = ");
int a = input.nextInt();
int b;
switch (a) {
    case 1:
        a = 100;
        b = 10;
    case 2:
    case 3:
        a = 1500;
        b = 25;
        break;
    case 4:
        a = 1000000;
        b = 40;
    default:
        a = 0;
        b = 0;
}
IO.println("a = " + a + "    b = " + b);
```

Eingabe:**Ausgabe:**

a = 3

a = 1500 b = 25

a = 2

a = 1500 b = 25

a = 1

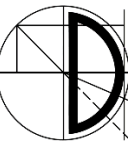
a = 1500 b = 25

a = 4

a = 0 b = 0

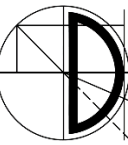
a = 7

a = 0 b = 0



Aufgabe: Schreiben Sie ein Programm, welches durch eine `switch`-Anweisung aus der Nummer eines Monats die Länge des betreffenden Monats ermittelt. Lassen Sie dabei Schaltjahre unberücksichtigt.

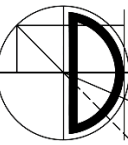
```
int monat = InputHelper.readInteger("Monat = ");
int tage = 0;
switch (monat) {
    case 4 :
    case 6 :
    case 9 :
    case 11: tage=30; break;
    case 2 : tage=28; break;
    default: tage=31; break;
}
IO.println("Tage = " + tage);
```

Worin unterscheiden sich die beiden Anweisungen?

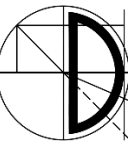
```
int punkte=InputHelper.readInteger("Punkte = ");
char note;
if (punkte >= 80)           note = '1';
else if (punkte >= 70)      note = '2';
else if (punkte >= 60)      note = '3';
else if (punkte >= 50)      note = '4';
else                        note = '5';
```

```
switch ((punkte / 10)) {
    case 10:
    case 9:
    case 8: note = '1'; break;
    case 7: note = '2'; break;
    case 6: note = '3'; break;
    case 5: note = '4'; break;
    default: note = '5'; break;
}
```



- Ist der Wert des `switch`-Ausdrucks während des Ablaufs einer `switch`-Anweisung unveränderlich?

```
int x = 1;  
switch(x) {  
    case 1: x = 3;  
    case 4: x = 5; break;  
    case 3: x = 22;  
}  
System.out.println(x) ; // Welchen Wert hat x?
```

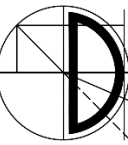


- In Java 14+ wurden einige neue Features für `switch`-Anweisungen freigeschaltet
- Diese können jetzt unter anderem in Zuweisungen verwendet werden und die Lambda-Notation (`->` Operator, siehe Kapitel 17) wird als kürzere Schreibweise verwendet
- Beispiel:

```
int a = 2;  
int b = switch(a) {  
    case 1 -> a/2;  
    case 2 -> a*2;  
    default -> a+2;  
}
```

Lambda-Notation

Ergebnis der Zuweisung wenn `a == 2`



Schon gewusst, was die Performance-Unterschiede zwischen `switch`- und `if`-Statements sind?

Im Gegensatz zu `if-else`-Anweisungen wird zur Laufzeit bei `switch`-Anweisungen nicht jeder Fall sequentiell überprüft.

Stattdessen wird vom Compiler eine Jump Table verwendet, die es erlaubt, direkt in den passenden `case` zu springen. Das macht `switch` effizienter, besonders wenn es viele Fälle gibt. Als Einschränkung kann `switch` in den meisten Programmiersprachen aber nur mit Datentypen verwendet werden, die einen kleinen Wertebereich haben. In Java werden z.B. ganzzahlige Datentypen bis `int` unterstützt, aber nicht `long`.

The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'J' and 'M' forming a square. The text 'Julius-Maximilians-' is in a smaller font above 'UNIVERSITÄT WÜRZBURG' which is in a large, bold, white sans-serif font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit